LAB

실습·리뷰·스터디 가이드

읽기에서 재구현·검증·설계 전이로

분석 대상
v2.1.88 reconstructed source

고정 리비전
2ca5ddabfed5

독자
스터디 운영자·참여자

사용 방식
워크북 · 평가

> 교육·연구용 파생 분석입니다. Anthropic의 공식 소스·공식 아키텍처 문서가 아닙니다.

정답집이 아니라 evidence workbook

각 실습은 목표, bounded file set, 조작, 기대 evidence, 중단 조건으로 구성한다.

구성	작성 내용	나쁜 예
질문	한 문장으로 falsifiable	“전체를 이해한다”
범위	파일 3-8개 + exact lines	src 전체
조작	mock/event/input 변화	그냥 읽기
관찰	trace/test/frame/graph	느낌
결론	사실/추정/한계 분리	영상과 같았다
중단	시간·범위·안전 조건	끝날 때까지

저작권 원칙

실습 저장소에 원 소스의 큰 덩어리를 복사하지 않는다. 작은 독립 toy implementation과 line reference를 사용한다.

프로비넌스와 신뢰도 카드

코드를 읽기 전에 분석 대상을 정확히 고정한다.

1. remote URL, commit, commit date, version 표기를 기록한다.
2. LICENSE 파일 유무와 README disclaimer를 분리한다.
3. QUICKSTART의 build limitation과 missing module 주장을 등급 B로 기록한다.
4. 공식 Claude Code repository/docs와 이 reconstructed source를 구분한다.
5. 분석에서 실행하지 않을 script와 이유를 적는다.

실습 · source card

한 페이지 provenance card를 만든다.

완료 증거: revision·scope·limitations·legal note·sources

코드 분포와 hotspot 재현

라인 수 대신 구조 신호를 계산한다.

1. TS compiler API로 .ts/.tsx를 parse한다.
2. directory별 files/lines, syntax feature count를 만든다.
3. 상대 import를 resolve해 fan-in/out과 SCC를 계산한다.
4. REPL.tsx, Tool.ts, query.ts를 graph에 표시한다.
5. metric의 한계—barrel, dynamic import, missing module—를 적는다.

실습 · metric audit

이 묶음의 source-metrics.json과 자신의 결과를 비교한다.

완료 증거: 차이 원인 세 가지

async generator 상태기계

문법 이해를 cancellation·error semantics까지 확장한다.

case	producer	consumer action	기대
normal	1,2,3 yield	끝까지 iterate	done true + terminal
slow consumer	10ms produce	100ms await	pull-based 순서 유지
consumer return	무한 producer	3개 후 break	finally/dispose
throw	2 뒤 error	try/catch	error propagation
abort	await 중 signal	abort()	bounded exit
delegation	child generator	parent yield*	return value 전달

실습 · protocol log

next/return/throw 호출과 iterator result를 모두 기록한다.

완료 증거: 상태 전이 표

축약 agent loop

모델 mock과 tool mock으로 두 번의 request를 만든다.

목표 trace

```
request #1 → text("검색") → tool_use(read, id=A)
tool A → tool_result(id=A, "found")
request #2 → text("요약") → completed
```

- ☐ 각 event에 sequence와 timestamp가 있다.
- ☐ tool_use/result id가 일치한다.
- ☐ unknown tool은 모델이 볼 수 있는 error result가 된다.
- ☐ maxTurns가 loop를 닫는다.
- ☐ AbortSignal이 model/tool에 모두 전달된다.
- ☐ terminal reason을 boolean 대신 union으로 표현한다.

fallback과 tombstone

append-only UI가 왜 충분하지 않은지 실험한다.

1. attempt A가 text와 tool_use 일부를 yield한 뒤 실패하게 한다.
2. attempt B가 fresh message를 생성한다.
3. tombstone 없이 transcript를 렌더해 orphan/duplicate를 관찰한다.
4. tombstone event를 추가해 materialized view가 old message를 제거하게 한다.
5. event log 자체는 append-only, view는 retractable인 구조를 설명한다.

실습 · view reducer

event[]에서 현재 messages를 만드는 pure reducer를 작성한다.

완료 증거: fallback 전후 snapshot test

Tool contract vertical slice

FileRead 또는 작은 toy tool을 schema부터 renderer까지 추적한다.

관찰 지점	기록
definition	name, schema, output, progress
classification	read-only, destructive, concurrency
validation	path/limits/context
permission	rule/mode/human
execution	signal/timeout/effect
normalization	tool_result shape
projection	compact/verbose/headless

실습 · one-page contract

한 tool의 모든 contract를 표 한 장으로 만든다.

완료 증거: 각 행에 source line ref

권한 결정표

mode, rule, tool-specific check, hook, human prompt의 precedence를 작은 모델로 재구성한다.

입력	mode	rule	hook	기대
read cwd	default	allow	none	allow
write cwd	default	ask	none	prompt
write cwd	dontAsk	ask	none	deny
shell dangerous	bypass	allow	block	block 여부 검증
outside root	acceptEdits	allow?	none	tool validation 우선
MCP delete	default	server hint readOnly	none	hint 불신

목표

표의 기대를 추측으로 채우지 말고 실제 resolver line과 test로 확인한다. 미확인은 미확인으로 남긴다.

PromptInput finite-state model

early return 연쇄를 transition table로 바꾼다.

1. Enter 처리부에서 guard condition을 순서대로 추출한다.
2. 각 guard를 state + event → action으로 표현한다.
3. 동시에 참일 수 있는 guard와 precedence를 찾는다.
4. normal submit까지 가는 minimal state를 만든다.
5. table-driven test로 branch coverage를 만든다.

실습 · guard refactor proposal

행동은 바꾸지 않고 guard 평가를 pure function으로 떼는 설계를 제안한다.

완료 증거: before/after dependency와 rollback

React/Ink frame 측정

"React가 알아서 batching"을 계측 가능한 주장으로 바꾼다.

측정	도구	질문
chunk arrival	mock async generator	입력 속도는?
state updates	instrumented setter	몇 번 호출?
renders	counter/profiler	몇 component가 render?
terminal writes	mock stdout	몇 회/몇 byte?
visual result	pseudo-TTY snapshot	누락/flicker/scrollback?

실습 · rate sweep

10/30/100/300 chunk·초에서 표를 만든다.

완료 증거: 성능 switch point와 한계

컨텍스트 예산 시뮬레이션

compaction을 품질·비용·회복 문제로 다룬다.

WORKLOAD A

긴 text 대화

요약 손실과 token 절감 비교.

WORKLOAD B

큰 tool result

persist/preview와 재읽기 비용.

WORKLOAD C

반복 file edits

attachment 중복과 stale context.

WORKLOAD D

media 포함

size error recovery 경로.

실습 · quality budget

각 workload에서 keep/snip/microcompact/summary를 언제 선택할지 policy를 만든다.

완료 증거: decision table + counterexample

MCP server 최소 구현과 trust test

tool discovery와 실행, result validation, human confirmation을 분리한다.

1. read-only toy tool 한 개를 tools/list로 노출한다.
2. input/output schema를 정의하고 invalid result를 일부러 반환한다.
3. client가 output schema를 검증하는지 확인한다.
4. server annotation에 거짓 readOnlyHint를 넣고 권한 경계가 속지 않는지 확인한다.
5. timeout/abort와 transport disconnect를 시험한다.

실습 · trust matrix

trusted/untrusted server × read/write tool × interactive/headless를 평가한다.

완료 증거: 12-cell policy matrix

Prompt injection 위협 모델

repository comment나 web content가 instruction처럼 보일 때 effect 경로를 추적한다.

공격 지점	원하는 오용	구조 통제
README	shell 실행 유도	instruction provenance + permission
web page	secret exfiltration	content isolation + network allowlist
MCP result	다른 tool 호출 유도	untrusted result + least privilege
plugin skill	지속 instruction	install trust + scope + review
tool output	fake success/authority	typed evidence + canonical readback

안전

실제 secret·외부 account·파괴적 명령을 사용하지 않는다. toy filesystem과 fake token으로 시뮬레이션한다.

의존성 cycle 한 조각 자르기

거대 SCC 전체가 아니라 하나의 vertical slice에서 seam을 찾는다.

1. Tool.ts와 FileReadTool/UI/runtime 주변 local graph를 추출한다.
2. type-only, runtime, UI, registry edge를 색칠한다.
3. UI render contract를 별도 adapter로 이동하는 가설을 세운다.
4. headless consumer와 interactive consumer의 public surface를 설계한다.
5. 변경 파일·compatibility·validation·rollback을 작성한다.

실습 · ADR 한 장

현 구조 유지 vs render adapter 분리의 trade-off를 비교한다.

완료 증거: 선택·근거·반대 증거·재검토 조건

agent runtime 변경 리뷰 12문항

기능이 동작하는지만 보지 않고 stream·취소·권한·정리·관측 가능성의 계약이 함께 보존되는지 확인한다.

- ☐ 새 event/terminal variant가 exhaustive하게 처리되는가?
- ☐ streaming 중 partial state가 실패 시 retract/settle되는가?
- ☐ AbortSignal이 모든 blocking boundary에 전달되는가?
- ☐ permission 검사와 실제 effect input이 동일한 canonical value인가?
- ☐ read-only·destructive·concurrency 분류가 input별로 정확한가?
- ☐ retry/loop/budget에 상한과 failure fingerprint가 있는가?
- ☐ tool_use/result id와 순서 불변식이 보존되는가?
- ☐ UI projection이 API-bound message를 mutate하지 않는가?
- ☐ feature gate 조합에서 missing import/untested path가 생기지 않는가?
- ☐ telemetry가 path/content/secret을 최소화하는가?
- ☐ headless/interactive/remote surface의 의미가 일치하는가?
- ☐ 공식 사양과 소스 추정이 문서에서 구분되는가?

개념 확인 질문

정답보다 근거와 반례를 말한다.

1. async generator와 ReadableStream은 어떤 요구에서 서로 대체 가능하고 불가능한가?
2. yield*가 하위 generator의 terminal을 어떻게 전달하는가?
3. tool result error와 thrown error를 왜 구분하는가?
4. React reconciler와 Ink renderer의 책임을 각각 한 문장으로 정의하라.
5. Static history가 없는 terminal chat UI에서 어떤 현상이 생길 수 있는가?
6. isReadOnly=true가 concurrency-safe를 보장하지 않는 반례를 들어라.
7. bypassPermissions가 sandbox를 대체하지 못하는 이유는?
8. MCP tool annotation을 권한 근거로 신뢰하면 안 되는 이유는?

소스 추적 질문

각 답에는 최소 두 개의 source line ref가 필요하다.

1. query loop가 tool_use continuation을 결정하는 실제 evidence는 무엇인가?
2. fallback 뒤 orphan message를 제거하는 event는 무엇인가?
3. tool concurrency 기본 상한과 보수적 parse failure 처리 위치는?
4. permission mode의 외부/내부 집합은 어떻게 다른가?
5. Tool builder가 채우는 기본 method와 그 보안 의미는?
6. memory prefetch가 조기 generator 종료에서 정리되는 표현은?
7. REPL onQuery의 try/finally가 복구하는 UI 상태는 무엇인가?
8. build script를 실행하지 않은 이유를 source limitation으로 설명하라.

20분 발표를 구성하는 법

기능 나열 대신 하나의 설계 긴장을 중심으로 구성한다.

시간	내용	예시
2분	사용자 사건	Enter 후 첫 token까지
3분	도메인 요구	점진성·취소·권한
5분	언어 메커니즘	async generator + AbortSignal
5분	코드 구조	query → tool runtime → projection
3분	실패/반례	fallback tombstone
2분	전이 가능한 원칙	append-only log vs retractable view

> **발표 금지**
> 파일 트리 낭독, 20줄 넘는 코드 캡처, 근거 없는 “Anthropic은 ~하려고 했다”식 의도 추정.

학습 성취 루브릭

파일을 찾는 수준에서 시작해 근거를 연결하고, 반증 가능한 설계 판단과 재사용 가능한 분석 프레임까지 발전시킨다.

수준	설명	증거
1 · 식별	파일/문법을 찾는다	anchor link
2 · 설명	기능 흐름을 말한다	sequence diagram
3 · 연결	요구↔메커니즘↔구조를 잇는다	concept map
4 · 검증	반례·실패를 실험한다	trace/test
5 · 전이	다른 agent runtime에 적용한다	design proposal

팀 평가는 사람의 암기량이 아니라 artifact의 재현성·근거 품질·불확실성 관리에 둔다.

다음 판에서 확장할 분석

이번 자료는 넓은 지도를 제공한다. 다음 연구는 exact revision과 실행 환경을 새로 고정해야 한다.

RUNTIME

실제 trace 계측

공식 binary의 event/latency를 source 가설과 비교.

VERSIONS

버전 diff

2.1.88 이후 query/tool/permission 변화 추적.

TESTING

복원 test corpus

source에 빠진 test를 behavior fixture로 재구성.

SECURITY

adversarial evaluation

prompt injection·path·MCP trust benchmark.

ARCHITECTURE

cycle decomposition

largest SCC를 vertical slice별로 분해.

PERFORMANCE

TUI benchmark

event rate→render/write/latency 관계 계측.

별도 작업으로 남김

공식 제품 binary 실행, 인증 계정 사용, 외부 시스템 효과, 대규모 소스 재배포는 이번 산출물 범위에 포함하지 않았다.

스터디를 마치기 전

아래 질문에 모두 “예”라면 이 프로젝트를 다른 코드베이스의 설계 도구로 사용할 수 있다.

- ☐ 코드와 문서, 공식 사양, 내 추정을 구분했다.
- ☐ 성공 경로와 실패·취소·거부 경로를 같은 깊이로 읽었다.
- ☐ 문법을 제품 요구와 연결했다.
- ☐ 한 vertical slice를 schema부터 UI까지 추적했다.
- ☐ 최소 하나의 설명을 실험으로 반증하려 했다.
- ☐ 성능 주장을 계측 지표로 바꿨다.
- ☐ 권한과 capability, effect와 evidence를 구분했다.
- ☐ 대형 소스 조각을 복제하지 않고 파생 지식을 만들었다.
- ☐ 다음 연구 범위와 미확인 사실을 남겼다.
- ☐ 다른 agentic CLI에 동일한 분석 프레임을 적용할 수 있다.

> **“흐름을 따라가는 것”에서 끝내지 않고, 왜 이 표현이어야 했는지와 어디서 깨지는지를 설명하는 것이 목표다.**

핵심 1차 자료

보고서 본문은 아래 소스의 짧은 참조와 파생 설명으로 구성했다.

범주	자료
분석 대상	stupidhumans/claude-code-source-code
공식 제품	anthropics/claude-code
언어	ECMAScript specification
타입	TypeScript Handbook — Narrowing
UI	Ink
SSE 배경	WHATWG SSE
MCP	MCP Tools specification
Hooks/permissions	Claude Code Hooks reference
Agent 연구	ReAct · Toolformer
참고 영상	async generator · React/Ink